

Projet OCR

liam.corbin

thibault.daguier

matthieu.oursin

gustave.molina

December 2025

Table des matières

1	Introduction	4
2	Organisation du groupe et répartition des rôles	5
2.1	Organisation de l'équipe	5
2.2	Répartition des tâches	5
3	Objectifs du projet et état d'avancement du projet	6
3.1	Objectif du projet	6
3.2	Avancement du projet	7
4	Choix technologiques, difficultés rencontrées et solutions mises en œuvre	8
4.1	Prétraitement	8
4.1.1	Rotation de l'image	11
4.1.2	Debruitage	11
4.2	Découpage de l'image	14
4.2.1	Détection	14
4.2.2	Découpage des lettres	16
4.2.3	Découpage liste de mots	17
4.2.4	Principe de l'Histogramme de Projection Horizontal . .	18
4.2.5	Algorithme de découpage	18
4.2.6	Rognage automatique	19
4.2.7	Sauvegarde et gestion des fichiers	19
4.3	Réseau Neurones	20
4.3.1	Architecture du réseau de neurones	20
4.3.1.1	Fonction de coût et rétropropagation	20
4.3.1.2	Entraînement sur lettres isolées	21
4.3.1.3	Entraînement à partir de mots segmentés . .	21
4.3.1.4	Sauvegarde et réutilisation du modèle	21
4.3.1.5	Utilisation du réseau en phase de reconnaissance	22

4.3.1.6	Reconnaissance des caractères de la grille . . .	22
4.3.1.7	Reconnaissance des caractères de mots	22
4.3.2	Implémentation et Entraînement du Réseau de Neurones	23
4.3.2.1	Essai initial sur EMNIST	23
4.3.3	Difficultés techniques rencontrées	24
4.3.3.1	Gestion de la mémoire et structure complexe	24
4.3.3.2	Instabilités numériques de la fonction Sigmoid	24
4.3.3.3	Complexité de la rétropropagation avec Soft- max	24
4.3.3.4	Problèmes liés à l'initialisation aléatoire . . .	24
4.3.3.5	Synthèse et solutions	25
4.4	Résolution de la grille	26
4.5	Interface	27
4.5.1	Etape 1	27
4.5.2	Etape 2	29
4.5.3	Etape 3	30
4.5.4	Etape 4	31
4.6	Makefile et gestion de la compilation	33
4.6.1	Présentation générale	33
4.6.2	Organisation des cibles	33
4.6.3	Gestion des fichiers source et des options de compilation	33
5	Conclusion	34
6	Bibliographie	36
6.1	Methode D'Otsu	36
6.2	Transformée de Hough	36
6.3	Librairie Gtk	36
6.4	Livre sur les neural networks	36
6.5	Documentation GdkPixBuf	36

1 Introduction

Dans ce rapport, nous allons vous présenter dans un premier temps les rôles et les missions de chaque membre du groupe, ce qu'il a fait pour le groupe, ce qu'il devait faire pour cette soutenance et quelles tâches il a réalisées.

Ensuite, dans un deuxième temps, nous vous parlerons des objectifs attendus lors de cette soutenance, de la répartition des tâches entre les membres du groupe pour réussir à atteindre ces objectifs, et si nous avons réussi à les atteindre, c'est-à-dire l'avancement réel du projet ainsi que les éventuels retards ou avancées que nous avons réalisés.

Enfin, dans un dernier temps, nous vous expliquerons, pour chaque fonctionnalité attendue, les choix technologiques réalisés, comme le choix des bibliothèques utilisées, les difficultés rencontrées et les solutions que nous avons trouvées pour résoudre ces problèmes et permettre le bon fonctionnement de chaque fonctionnalité. Nous détaillerons aussi les différents algorithmes utilisés et que nous avons dû appliquer pour chaque fonctionnalité.

En conclusion, ce rapport vous donnera une vue d'ensemble du projet et des fonctionnalités réalisées.

2 Organisation du groupe et répartition des rôles

2.1 Organisation de l'équipe

L'organisation du travail au sein de l'équipe constituant l'un des éléments les plus importants pour la réussite du projet, plusieurs outils collaboratifs ont été mis en place afin de structurer la communication et le suivi des tâches. Un serveur Discord a été utilisé comme principal moyen d'échange, permettant de discuter des choix techniques, de signaler les difficultés rencontrées et de coordonner l'avancement des différentes parties du projet.

La gestion du code source a été assurée à l'aide de GitHub, permettant un suivi des modifications, une collaboration efficace entre les membres du groupe et une intégration progressive des fonctionnalités développées.

Par ailleurs, un agenda partagé a été mis en place afin de permettre à chaque membre de l'équipe de suivre les échéances importantes du projet, telles que les dates de rendus, les soutenances et les objectifs intermédiaires. Des réunions de groupe régulières ont complété cette organisation, offrant un cadre propice aux échanges, à la planification des tâches et aux prises de décisions collectives.

2.2 Répartition des tâches

Le projet a été réalisé par un groupe de quatre personnes, avec une répartition des rôles définie dès le début. Cette répartition a été amenée à être légèrement modifiée par rapport à nos premières estimations. Les principales raisons sont que pour certaines tâches être à plusieurs n'était pas nécessaire et aussi pour mieux séparer les charges de travail.

Donc tout d'abord Oursin Matthieu qui s'est principalement chargé du développement de l'interface graphique ainsi que des étapes de prétraitement de

l'image. Ensuite Daguier Thibault qui a été responsable de la conception et de l'implémentation du réseau de neurones dédié à la reconnaissance des caractères. Enfin la détection et le découpage de la grille ont été pris en charge par Molina Gustave, tandis que Corbin Liam s'est occupé du découpage des mots de la liste associée à la grille.

Cette organisation a permis de travailler en parallèle sur des modules complémentaires tout en facilitant leur intégration progressive dans l'application finale.

3 Objectifs du projet et état d'avancement du projet

3.1 Objectif du projet

L'objectif du projet OCR est de concevoir une application capable de résoudre automatiquement une grille de mots croisé à partir d'une image. Le programme prend en entrée une image représentant une grille accompagnée de sa liste de mots, et produit en sortie une version résolue de cette grille, mettant en évidence les mots recherchés.

Le programme doit être capable de traiter des images de qualités variables, allant d'images parfaitement propres à des images présentant des défauts tels que des reliefs, un contraste faible ou une rotation. L'objectif est donc de concevoir des algorithmes suffisamment robustes pour gérer ces imperfections tout en respectant les contraintes imposées.

Enfin, le projet vise à fournir une application cohérente, intégrant une interface graphique permettant à l'utilisateur de charger une image, d'en visualiser le traitement, et d'obtenir une grille de mots cachés résolue.

Au-delà de ses objectifs techniques, ce projet vise également à développer les compétences liées au travail d'équipe. La réalisation d'une application complète, combinant traitement d'image, reconnaissance de caractères, algorithmique et interface graphique, nécessite une répartition claire des tâches et une bonne communication entre les membres du groupe. Chaque sous-partie du projet étant étroitement liée aux autres, une communication régulière entre les différents responsables de fonctionnalités est très importante. Une bonne organisation permet de progresser simultanément sur plusieurs parties du projet tout en assurant leur compatibilité lors de l'intégration.

3.2 Avancement du projet

À ce stade du projet, l'ensemble des fonctionnalités principales a été implémenté, permettant de disposer d'une chaîne de traitement cohérente allant du chargement de l'image jusqu'à l'affichage du résultat final. Le projet est globalement fonctionnel pour des images de niveaux 1 et 2 du cahier des charges. Concernant le prétraitement de l'image, les étapes nécessaires au traitement des images jusqu'au niveau 2 sont opérationnelles. La conversion en niveaux de gris ainsi que la binarisation ont été implémentées et permettent de simplifier l'analyse des images. La rotation automatique de l'image est également fonctionnelle.

La partie détection et découpage est entièrement fonctionnelle. La détection de la grille permet d'identifier correctement la zone correspondant à la grille de mots. Le découpage des lettres de la grille ainsi que le découpage de la liste de mots sont opérationnels et produisent des sous-images exploitables par les modules suivants.

Le réseau de neurones, . . .

La résolution de la grille est fonctionnelle via le solver qui est capable de rechercher un mot dans une grille fournie et de retourner ses coordonnées, pour pouvoir être exploiter par la suite.

Enfin, l'interface graphique est opérationnelle. Elle permet de charger une image, d'afficher les différentes étapes intermédiaires du traitement, ainsi que le résultat final de la grille résolue. Cette interface assure l'intégration des différentes composantes du projet et facilite la démonstration du fonctionnement global de l'application.

Dans l'ensemble, le projet présente un état d'avancement satisfaisant. Les fonctionnalités principales sont en place et intégrées, et les efforts actuels portent principalement sur l'amélioration et la stabilisation du réseau de neurones afin d'obtenir une reconnaissance de caractères plus fiable.

4 Choix technologiques, difficultés rencontrées et solutions mises en œuvre

4.1 Prétraitement

L'objectif du prétraitement était de convertir l'image en noir et blanc afin d'améliorer sa lecture par le réseau de neurones, mais également de la faire pivoter automatiquement en fonction de l'angle de la grille pour les images de niveau 2 qui en avaient besoin. Ces fonctionnalités ont été mises en place avec succès. Nous allons donc détailler les méthodes utilisées pour atteindre ces objectifs, ainsi que les difficultés rencontrées lors de leur développement.

Dans un premier temps, l'ensemble des fonctions liées au prétraitement de l'image a été regroupé dans le fichier `image.c`, dont le nom est explicite. Ce fichier contient plusieurs fonctions, dont l'une appelée `binarize-otsu`, qui applique la méthode de traitement d'image d'Otsu. Cette méthode, très connue et reconnue pour sa fiabilité, nous a permis de binariser l'image fournie par l'interface en déterminant, à partir d'un seuil, si chaque pixel devait être considéré comme noir ou blanc.

H	C	A	M	P	A	I	G	N	S	L	Z
Y	C	P	H	Q	M	L	C	A	B	E	E
U	W	O	R	K	D	R	I	V	E	K	T
K	O	Y	O	D	Q	U	G	Y	E	A	F
S	A	L	E	S	I	Q	P	R	W	S	Q
E	T	U	M	B	L	J	P	E	Z	X	C
D	M	A	I	L	C	A	Y	N	U	O	R
P	R	K	Y	U	G	P	E	O	P	L	E
H	U	H	G	B	O	A	V	J	L	M	A
P	K	Y	H	A	D	C	R	M	R	S	T
A	L	M	U	L	A	A	U	X	S	T	O
K	R	U	B	O	O	K	S	A	T	O	R

CAMPAIGNS

ULAA

CRM

ONE

DESK

SURVEY

CREATOR

BOOKS

PEOPLE

MAIL

SALESIQ

WORKDRIVE

CLIQ

FIGURE 1 – Grille avant la méthode otsu

H	C	A	M	P	A	I	G	N	S	L	Z
Y	C	P	H	Q	M	L	C	A	B	E	E
U	W	O	R	K	D	R	I	V	E	K	T
K	O	Y	O	D	Q	U	G	Y	E	A	F
S	A	L	E	S	I	Q	P	R	W	S	Q
E	T	U	M	B	L	J	P	E	Z	X	C
D	M	A	I	L	C	A	Y	N	U	O	R
P	R	K	Y	U	G	P	E	O	P	L	E
H	U	H	G	B	O	A	V	J	L	M	A
P	K	Y	H	A	D	C	R	M	R	S	T
A	L	M	U	L	A	A	U	X	S	T	O
K	R	U	B	O	O	K	S	A	T	O	R

CAMPAIGNS
 ULAA
 CRM
 ONE
 DESK
 SURVEY
 CREATOR
 BOOKS
 PEOPLE
 MAIL
 SALESIQ
 WORKDRIVE
 CLIQ

FIGURE 2 – Grille après la méthode otsu

4.1.1 Rotation de l'image

Ensuite, comme expliqué plus haut, pour certaines grilles qui n'étaient pas droites, il a fallu effectuer une rotation automatique de l'image afin qu'elle devienne droite et facilite la détection des lettres par le réseau de neurones. Pour cela, nous avons créé deux fonctions : `detect-skew-angle` et `rotate-pixbuf`.

La seconde est plutôt simple et était déjà présente lors de la première soutenance. Elle réalise simplement une rotation de l'image d'un nombre de degrés donné en argument. Cette fonction effectue la rotation à l'aide des bibliothèques GTK et Cairo, qui simplifient grandement le travail.

En revanche, la partie la plus complexe concernait la détection de l'angle, réalisée par la fonction `detect-skew-angle`. Celle-ci détecte l'angle en s'appuyant sur une technique elle aussi très fiable et bien connue : la transformée de Hough. Cette méthode de reconnaissance d'image permet de détecter les lignes de la grille présentes dans l'image et ainsi de déterminer l'angle nécessaire à appliquer à l'image.

4.1.2 Debruitage

Certaines images présentaient un niveau de bruit très élevé, dû à de nombreuses imperfections telles que des pliures ou de multiples petits points, ce qui dégradait fortement la qualité visuelle. Il était donc indispensable de supprimer ces imperfections afin de pouvoir poursuivre correctement les étapes suivantes du traitement de l'image, sans quoi la suite du processus aurait été compromise.

Cette phase s'est révélée particulièrement complexe, car un mauvais réglage entraînait soit la suppression excessive de pixels, ce qui dégradait fortement la qualité des lettres en créant des trous et rendait leur reconnaissance par le réseau de neurones très difficile, soit un épaississement excessif des lettres, qui n'éliminait pas efficacement le bruit et empêchait d'atteindre l'objectif recherché.

Il était donc nécessaire de trouver un juste milieu, en retirant suffisamment de pixels pour réduire le bruit tout en préservant la qualité de l'image. Nous avons finalement réussi à trouver un équilibre satisfaisant permettant d'éliminer l'essentiel du bruit.

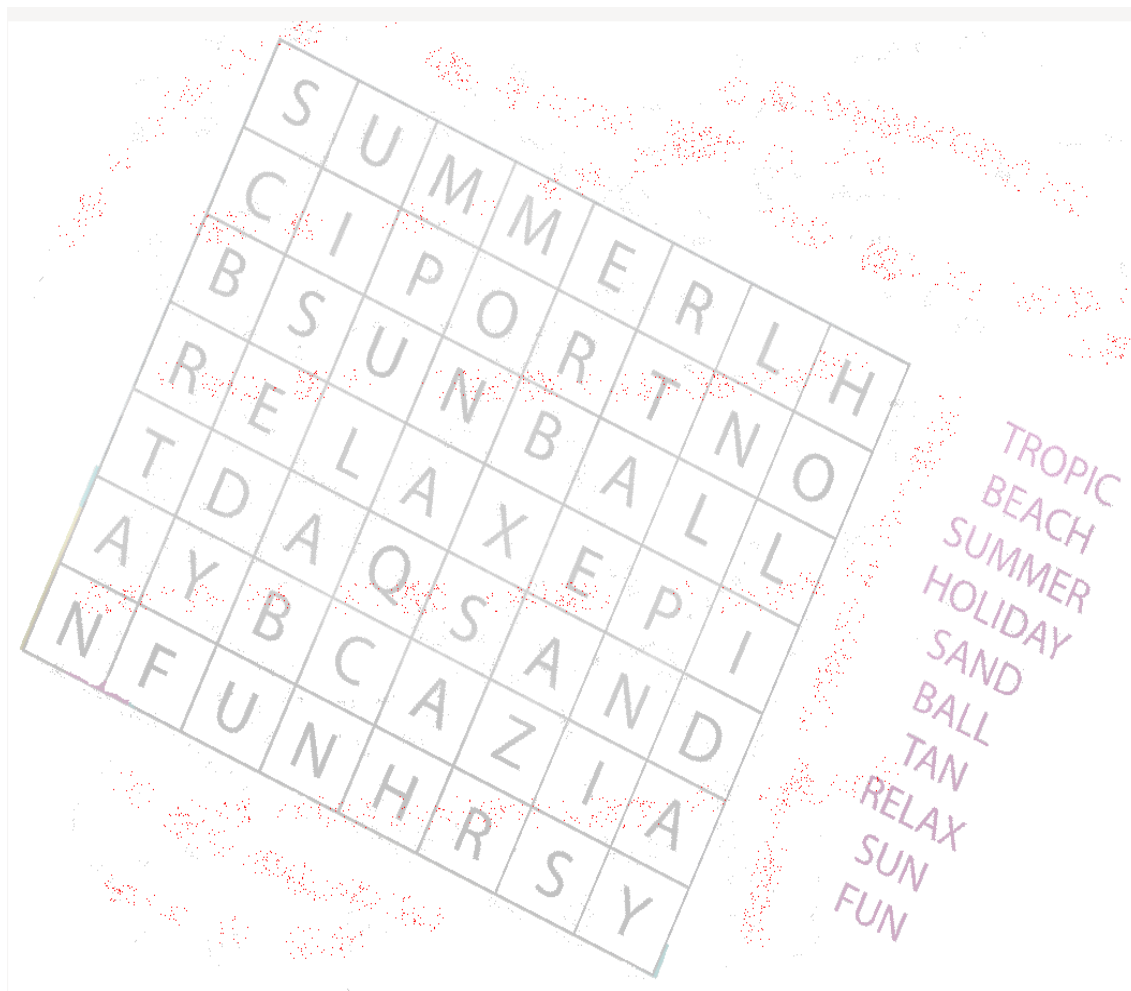


FIGURE 3 – Grille avant le débruitage et la rotation automatique

S	U	M	M	E	R	L	H
C	I	P	O	R	T	N	O
B	S	U	N	B	A	L	L
R	E	L	A	X	E	P	I
T	D	A	Q	S	A	N	D
A	Y	B	C	A	Z	I	A
N	F	U	N	H	R	S	Y

TROPIC
 BEACH
 SUMMER
 HOLIDAY
 SAND
 BALL
 TAN
 RELAX
 SUN
 FUN

FIGURE 4 – Grille après le débruitage et la rotation automatique

4.2 Découpage de l'image

4.2.1 Détection

La détection de la grille vise à localiser automatiquement, au sein de l'image d'entrée prétraitée, la zone correspondant à la grille de mots cachés. Cette étape constitue un point clé de la chaîne de traitement, car elle conditionne directement la qualité du découpage des lettres et, par conséquent, la fiabilité de la reconnaissance de caractères. Une détection incorrecte de la grille entraînerait des erreurs propagées sur l'ensemble des étapes suivantes. L'objectif principal de cette phase est de distinguer la grille de la liste de mots ainsi que des autres éléments visuels éventuellement présents dans l'image. Pour cela, la méthode retenue repose sur une analyse de la distribution des pixels non blancs, en s'appuyant sur les fonctionnalités fournies par la bibliothèque GdkPixbuf, utilisée pour l'accès aux données de l'image et la manipulation des pixels. Un seuil de luminosité est appliqué afin de différencier le fond clair du contenu utile, ce qui permet de ne considérer que les pixels appartenant aux traits de la grille ou aux caractères.

L'analyse débute par le calcul d'une projection verticale de l'image à l'aide de la fonction `calculate-vertical-projection`. Cette fonction parcourt l'image colonne par colonne et comptabilise, pour chacune d'elles, le nombre de pixels ne correspondant pas au fond. Le résultat est stocké sous la forme d'un tableau représentant la densité verticale du contenu visuel. Cette représentation permet de mettre en évidence les zones fortement chargées en information, ainsi que les zones plus vides.

À partir de cette projection, la fonction `find-widest-gap` est utilisée afin d'identifier le plus large intervalle de colonnes présentant une faible densité de pixels. Cet intervalle correspond généralement à l'espace séparant la grille de la liste de mots. Ce choix repose sur l'hypothèse que ces deux éléments sont distincts spatialement et séparés par une zone relativement vide dans l'image, hypothèse vérifiée pour les images de niveaux 1 et 2 du ca-

hier des charges. Une fois cet espace identifié, l'image est découpée en deux zones candidates à l'aide de la fonction `gdk-pixbuf-new-subpixbuf`. Afin de déterminer laquelle de ces zones correspond effectivement à la grille, une projection horizontale est calculée sur chacune d'elles à l'aide de la fonction `calculate-horizontal-projection`. Cette projection permet d'analyser la répartition verticale du contenu et de mettre en évidence la structure plus dense et régulière caractéristique de la grille. La fonction `count-empty-lines` est ensuite utilisée pour compter le nombre de lignes peu denses dans chaque zone. La zone présentant une structure régulière et une densité plus homogène est identifiée comme étant la grille, tandis que l'autre est associée à la liste de mots. Cette étape permet de confirmer le découpage effectué précédemment et d'éviter les erreurs de classification entre les différentes zones de l'image.

Cette méthode de détection, fondée sur l'utilisation de projections simples et de seuils fixes, a été privilégiée pour sa robustesse et sa faible complexité algorithmique. Elle permet de localiser efficacement les différents éléments de l'image tout en restant compatible avec les contraintes de performance et de simplicité imposées par le projet.

4.2.2 Découpage des lettres

Le découpage des lettres de la grille est assuré par la fonction `cut-grid-cells-gridlines`, dont le rôle est d'extraire individuellement chaque case de la grille à partir de la zone détectée précédemment. Cette fonction constitue le cœur du découpage des lettres, car elle assure la transition entre la détection de la grille et la reconnaissance de caractères. Elle s'appuie directement sur la structure géométrique régulière de la grille, caractérisée par un alignement uniforme des lignes et des colonnes.

Dans un premier temps, le découpage repose sur l'analyse des projections horizontales et verticales de l'image correspondant à la grille. Ces projections permettent de repérer les lignes séparant les cases en identifiant les zones où la densité de pixels non blancs est élevée. Cette étape est essentielle pour retrouver la structure interne de la grille et déterminer précisément les positions des lignes horizontales et verticales qui la composent.

À partir de ces informations, la fonction détecte les positions des lignes formant la structure de la grille. Une fois ces lignes identifiées, la fonction `cut-grid-cells-gridlines` parcourt la grille à l'aide de boucles imbriquées. Chaque intervalle délimité par deux lignes horizontales et deux lignes verticales correspond alors à une case de la grille. Cette approche permet de traiter la grille de manière systématique et de garantir un découpage cohérent sur l'ensemble de l'image. Pour chaque case identifiée, une sous-image est extraite à l'aide de la fonction `gdk-pixbuf-new-subpixbuf`, fournie par la bibliothèque Gdk-Pixbuf. Afin d'améliorer la qualité des images extraites, de légères marges sont retirées autour de chaque case. Cette opération permet d'éliminer les bordures de la grille tout en conservant principalement le caractère contenu dans la cellule, ce qui est essentiel pour améliorer les performances du module de reconnaissance de caractères. Chaque lettre est ensuite sauvegardée sous forme d'image indépendante, prête à être utilisée comme entrée du réseau de neurones.

Cette approche, fondée sur l'exploitation directe des lignes de la grille, ga-

rantit un découpage régulier et cohérent tant que le prétraitement de la grille est correctement réalisé. Elle offre un bon compromis entre simplicité d'implémentation, robustesse et qualité des résultats.

Lors du développement, un des problèmes rencontrés concernait la sensibilité de la détection des lignes aux variations d'épaisseur des bordures. Ce problème a été corrigé en utilisant des seuils relatifs, basés sur les valeurs maximales des projections horizontales et verticales. Cette solution permet de s'adapter automatiquement à différentes épaisseurs de traits et d'améliorer la robustesse globale de la détection des lignes.

Un autre problème est apparu dans certains cas où les étapes de prétraitement de l'image entraînaient une atténuation importante, voire une disparition partielle des traits de la grille. Afin de gérer cette situation, l'algorithme de détection a été adapté pour ne pas dépendre exclusivement de la présence de lignes continues, mais également de la densité globale des pixels et de la régularité de leur répartition. Cette adaptation permet de maintenir un découpage correct de la grille même lorsque ses contours sont peu visibles, voire partiellement absents.

4.2.3 Découpage liste de mots

Le découpage de la liste de mots a pour objectif d'isoler chaque mot de la liste associée à la grille, une fois la zone correspondante identifiée lors de la phase de détection. Contrairement au découpage de la grille, cette étape ne repose pas sur une structure bidimensionnelle régulière, mais sur une organisation essentiellement verticale des éléments textuels.

4.2.4 Principe de l'Histogramme de Projection Horizontal

Pour segmenter les lignes de texte, nous utilisons une technique d'analyse de densité de pixels appelée projection horizontale. L'algorithme parcourt l'image de la liste ligne par ligne (axe Y). Pour chaque ligne, il compte le nombre de pixels noirs (ceux qui ne sont pas considérés comme du fond blanc, défini par BACKGROUND-THRESHOLD mis à deux arbitrairement). Dans notre fonction calculate-horizontal-projection, nous générons un tableau où l'index correspond à la coordonnée Y et la valeur correspond au nombre de pixels "d'encre".

- Pics (valeurs élevées) : Indiquent la présence d'une ligne de texte.
- Creux (valeurs proches de 0) : Indiquent les espaces blancs entre les mots.

4.2.5 Algorithme de découpage

Le découpage s'effectue par une itération linéaire sur le tableau de projection horizontale. Nous utilisons un automate à états finis simple pour déterminer les frontières des mots :

- Seuillage : Nous définissons un seuil de bruit (PROJECTION-THRESHOLD, fixé à 2 dans notre code) pour ignorer les poussières ou artefacts mineurs.
- Détection de début de mot : Tant que la projection est inférieure au seuil, nous sommes dans un espace vide. Dès que la projection dépasse ce seuil, nous marquons la coordonnée word-start-y.
- Détection de fin de mot : Lorsque la projection redescend sous le seuil après avoir été au-dessus, nous considérons que le mot est terminé. La hauteur du mot est alors calculée ($y - \text{word-start-y}$).

Ce processus nous permet de découper des bandes horizontales contenant chacune un mot entier sur toute la largeur de la zone de liste.

4.2.6 Rognage automatique

Les bandes horizontales extraites contiennent beaucoup d'espace blanc superflu à gauche et à droite du mot. Pour optimiser l'image avant l'OCR et réduire la taille des fichiers, nous appliquons une fonction de rognage (`trim-whitespace`).

Cette fonction parcourt la sous-image pour trouver les pixels extrêmes (`min-x`, `max-x`, `min-y`, `max-y`) qui ne sont pas blancs. Elle redéfinit ensuite une "Bounding Box" stricte autour des caractères. Le résultat est une image propre, centrée sur le mot, sans marges inutiles.



FIGURE 5 – Exemple de mot après rognage

4.2.7 Sauvegarde et gestion des fichiers

Les images traitées sont stockées dans une liste chaînée (`GList *word-images`). Une fois le traitement terminé, le programme :

- Crée un répertoire dédié nommé "Words".
- Nettoie ce répertoire s'il contient d'anciens fichiers (`clean-directory`) pour éviter les conflits.
- Sauvegarde chaque élément de la liste à la suite sous la forme `mot-XX.png` en utilisant la fonction `save-pixbuf`.

4.3 Réseau Neurones

Cette partie présente la conception et l'utilisation d'un réseau de neurones artificiel destiné à la reconnaissance optique de caractères. Le réseau est utilisé pour identifier des lettres de l'alphabet à partir d'images, aussi bien sous forme de caractères isolés que de mots complets issus d'une segmentation préalable.

4.3.1 Architecture du réseau de neurones

Le réseau de neurones implémenté est un perceptron multicouche entièrement connecté, composé de trois couches fixe :

- une couche d'entrée de 784 neurones, correspondant à une image normalisée de 28×28 pixels ;
- une couche cachée de taille n fixé, permettant l'extraction de caractéristiques pertinentes ;
- une couche de sortie de 26 neurones, représentant les lettres de l'alphabet de A à Z.

La couche cachée utilise une fonction d'activation sigmoïde afin d'introduire une non-linéarité dans le modèle. La couche de sortie applique une fonction softmax, produisant une distribution de probabilité sur l'ensemble des classes possibles.

4.3.1.1 Fonction de coût et rétropropagation

L'apprentissage du réseau repose sur une fonction de coût de type entropie croisée catégorielle, adaptée aux problèmes de classification multiclasse. La sortie softmax permet d'interpréter les résultats du réseau comme des probabilités.

La mise à jour des paramètres est réalisée par rétropropagation du gradient, combinée à une descente de gradient stochastique. Les gradients sont calculés successivement pour la couche de sortie puis pour la couche cachée, avant d'être propagés jusqu'aux poids de la couche d'entrée. Un taux d'ap-

prentissage fixe est utilisé afin de contrôler l’amplitude des mises à jour.

4.3.1.2 Entraînement sur lettres isolées

Dans un premier temps, le réseau est entraîné à partir d’images de lettres isolées. Chaque image est associée à une étiquette correspondant à la lettre attendue. Les échantillons sont mélangés à chaque époque afin d’éviter les biais liés à l’ordre des données.

À chaque itération, une image est prétraitée, propagée dans le réseau, puis utilisée pour calculer l’erreur et mettre à jour les paramètres. Les performances sont évaluées à l’aide de la perte moyenne et du taux de reconnaissance, permettant de suivre la convergence du modèle au fil des époques.

4.3.1.3 Entraînement à partir de mots segmentés

Afin d’améliorer la robustesse du réseau face aux variations d’écriture et aux effets de segmentation, un second mode d’entraînement est utilisé à partir d’images de mots complets. Chaque mot est segmenté en lettres individuelles à l’aide d’une projection verticale, permettant d’identifier les zones contenant de l’encre.

Lorsque le nombre de segments détectés correspond à la longueur du mot attendu, chaque segment est considéré comme un échantillon valide. Les lettres extraites sont prétraitées de la même manière que les lettres isolées, puis utilisées pour enrichir le corpus d’entraînement. Cette approche permet au réseau d’apprendre à partir de données plus proches des conditions réelles d’utilisation.

4.3.1.4 Sauvegarde et réutilisation du modèle

À l’issue de l’entraînement, les poids et biais du réseau sont sauvegardés dans un fichier binaire. Ce fichier constitue un modèle réutilisable, pouvant être chargé ultérieurement sans nécessiter de nouvel apprentissage. Cette séparation entre apprentissage et utilisation permet de réduire considérablement le temps d’exécution lors de la reconnaissance.

4.3.1.5 Utilisation du réseau en phase de reconnaissance

Lors de la phase de reconnaissance, le réseau est utilisé exclusivement en mode inférence. Les paramètres appris sont chargés depuis le fichier de sauvegarde et aucune mise à jour n'est effectuée.

Chaque image d'entrée est prétraitée puis propagée dans le réseau par une propagation avant. La couche de sortie fournit une distribution de probabilité sur les lettres possibles, et la lettre prédite correspond à celle dont la probabilité est maximale.

4.3.1.6 Reconnaissance des caractères de la grille

Dans le cas d'une grille, chaque cellule est traitée indépendamment. Les images correspondant aux différentes cases sont chargées, prétraitées puis reconnues par le réseau. Les lettres obtenues sont ensuite écrites dans un fichier texte en respectant la structure de la grille d'origine.

4.3.1.7 Reconnaissance des caractères de mots

Pour la reconnaissance de mots, chaque image est d'abord segmentée en lettres individuelles à l'aide d'une projection verticale. Chaque segment est ensuite reconnu indépendamment par le réseau de neurones.

Les lettres prédites sont concaténées dans l'ordre de lecture afin de reconstituer le mot complet. Les résultats sont sauvegardés dans un fichier texte, permettant une exploitation ultérieure des données reconnues.

4.3.2 Implémentation et Entraînement du Réseau de Neurones

4.3.2.1 Essai initial sur EMNIST

Dans un premier temps, nous avons tenté d'utiliser la base de données *EMNIST* pour entraîner notre réseau de neurones. Cette base contient des images de lettres manuscrites, à la fois majuscules et minuscules, et elle est souvent utilisée pour des modèles MNIST-like. Cependant, cet essai s'est avéré peu efficace pour notre application spécifique. Plusieurs raisons expliquent cette inadéquation :

- Nos images contiennent systématiquement un *bord* ou un encadrement autour de chaque lettre, ce qui diffère des images EMNIST centrées et normalisées.
- Nous souhaitons uniquement reconnaître des lettres majuscules, alors que EMNIST contient à la fois majuscules et minuscules. L'entraînement sur ce corpus hétérogène ne permettait pas d'apprendre efficacement la distinction des lettres majuscules seules.

Suite à cet échec, nous avons choisi de renforcer l'apprentissage exclusivement sur des lettres majuscules, en utilisant des images provenant de nos propres jeux de données. Cette approche a permis de mieux adapter le réseau aux caractéristiques spécifiques de nos images et à la tâche ciblée.

4.3.3 Difficultés techniques rencontrées

Lors de la mise en œuvre du réseau de neurones, plusieurs obstacles majeurs ont été identifiés, affectant la stabilité et la performance du modèle.

4.3.3.1 Gestion de la mémoire et structure complexe

Le réseau est représenté par une structure contenant les poids, les biais et les sorties intermédiaires pour chaque couche. La manipulation de cette structure a causé de nombreux *core dumps* et erreurs de segmentation. Ces problèmes étaient principalement liés à des accès hors limites ou à des libérations de mémoire incorrectes. Il a été nécessaire de vérifier systématiquement chaque allocation, libération et initialisation de tableaux pour garantir la stabilité du programme.

4.3.3.2 Instabilités numériques de la fonction Sigmoid

La fonction Sigmoid, utilisée pour la couche cachée, a montré des limitations numériques. Pour de grandes valeurs d'entrée, l'utilisation de l'exponentielle a généré des valeurs *NaN*, perturbant l'apprentissage. Des bornes sur les entrées et une attention particulière à l'échelle des activations ont été nécessaires pour limiter ces problèmes.

4.3.3.3 Complexité de la rétropropagation avec Softmax

La couche de sortie utilisant Softmax couplée à la fonction de perte *cross-entropy* a nécessité une attention particulière. La dérivée combinée n'est pas triviale, et des erreurs initiales dans le calcul des gradients ont entraîné une convergence lente et des oscillations. Un suivi détaillé des gradients et des mises à jour de poids a été indispensable pour garantir un apprentissage correct.

4.3.3.4 Problèmes liés à l'initialisation aléatoire

L'initialisation classique des poids avec des valeurs uniformes sur un intervalle fixe a entraîné des comportements incohérents :

le réseau répondait systématiquement par la même lettre lors des tests, quelle que soit l'entrée. Cela est dû à la saturation de la Sigmoid et à un mauvais conditionnement initial. L'adoption de l'initialisation *Xavier*, adaptée aux dimensions des couches, a permis une meilleure distribution des activations et une convergence plus stable.

4.3.3.5 Synthèse et solutions

Ces difficultés ont mis en évidence l'importance de plusieurs facteurs pour un apprentissage stable :

- Une gestion rigoureuse de la mémoire et de la structure du réseau.
- La surveillance des instabilités numériques liées aux fonctions d'activation.
- Une compréhension précise de la rétropropagation, en particulier avec Softmax.
- L'utilisation d'une initialisation adaptée des poids (*Xavier*) pour éviter les saturations.

Grâce à ces ajustements, le réseau a pu être entraîné efficacement sur notre corpus de lettres majuscules, garantissant des performances adaptées à nos images et à notre tâche de reconnaissance.

4.4 Résolution de la grille

Pour l'algorithme de résolution de la grille, il a d'abord été nécessaire de convertir l'image en une matrice afin de pouvoir la parcourir efficacement. Cette conversion a été réalisée à l'aide d'une fonction chargée de lire le fichier d'entrée et de recopier chaque élément de chaque ligne dans la matrice correspondante. Cette étape est essentielle, car elle permet de manipuler les données de la grille de manière structurée et exploitable par l'algorithme.

Une fois la matrice construite, nous l'avons parcourue afin de détecter la première lettre du mot recherché. Lorsqu'une correspondance est trouvée, l'algorithme examine alors les huit directions possibles autour de cette lettre (horizontalement, verticalement et en diagonale) pour vérifier si les lettres suivantes du mot sont présentes dans l'une de ces directions. Si aucune correspondance valide n'est trouvée, le parcours de la matrice se poursuit jusqu'à la prochaine occurrence de la première lettre.

La phase d'analyse des huit directions s'est révélée relativement complexe, car il fallait anticiper tous les cas possibles et mémoriser précisément la position de la première lettre détectée. En effet, lorsqu'il existait plusieurs occurrences de la seconde lettre du mot autour de la première, il était nécessaire de gérer chacune de ces possibilités afin de ne pas manquer une solution valide. Cette gestion des différents cas a demandé une attention particulière lors de la conception de l'algorithme.

En conclusion, cette partie du projet a été l'une des plus délicates à concevoir sur le plan algorithmique. Cependant, une fois l'ensemble des cas correctement pris en compte et la structure de base solidement mise en place, la réalisation de l'algorithme s'est déroulée sans difficultés majeures et le programme fonctionne de manière satisfaisante.

4.5 Interface

L'interface de la grille a été entièrement réalisée à l'aide de la librairie GTK, c'est une librairie qui permet le développement d'interface graphique, c'est une librairie reconnue dans le milieu des interfaces graphiques.

En plus d'être très versatile dans ses utilisations, GTK est utilisée dans de nombreux logiciels professionnels comme GIMP, Firefox, LibreOffice et bien d'autres.

Encore un point positif de cette librairie est le fait qu'elle soit Open Source ! Pour en apprendre plus sur la librairie GTK et ses applications dans le développement et son histoire, vous pouvez aller voir la bibliographie !

Donc notre interface graphique sera composée de nombreux boutons mais au lancement elle n'en affichera que 2 car les boutons s'afficheront au fur et à mesure du traitement de l'image ainsi que le bouton sauvegarder qui restera tout du long.

4.5.1 Etape 1

Donc le premier bouton permettra de charger l'image, c'est-à-dire d'ouvrir le navigateur de fichiers et permettre à l'utilisateur de sélectionner son image et ensuite de la charger dans l'interface. L'image apparaîtra alors. Le bouton sauvegarde est classique, il permet d'ouvrir l'explorateur de fichiers pour sélectionner l'emplacement où sauvegarder l'image affichée à l'écran ensuite.

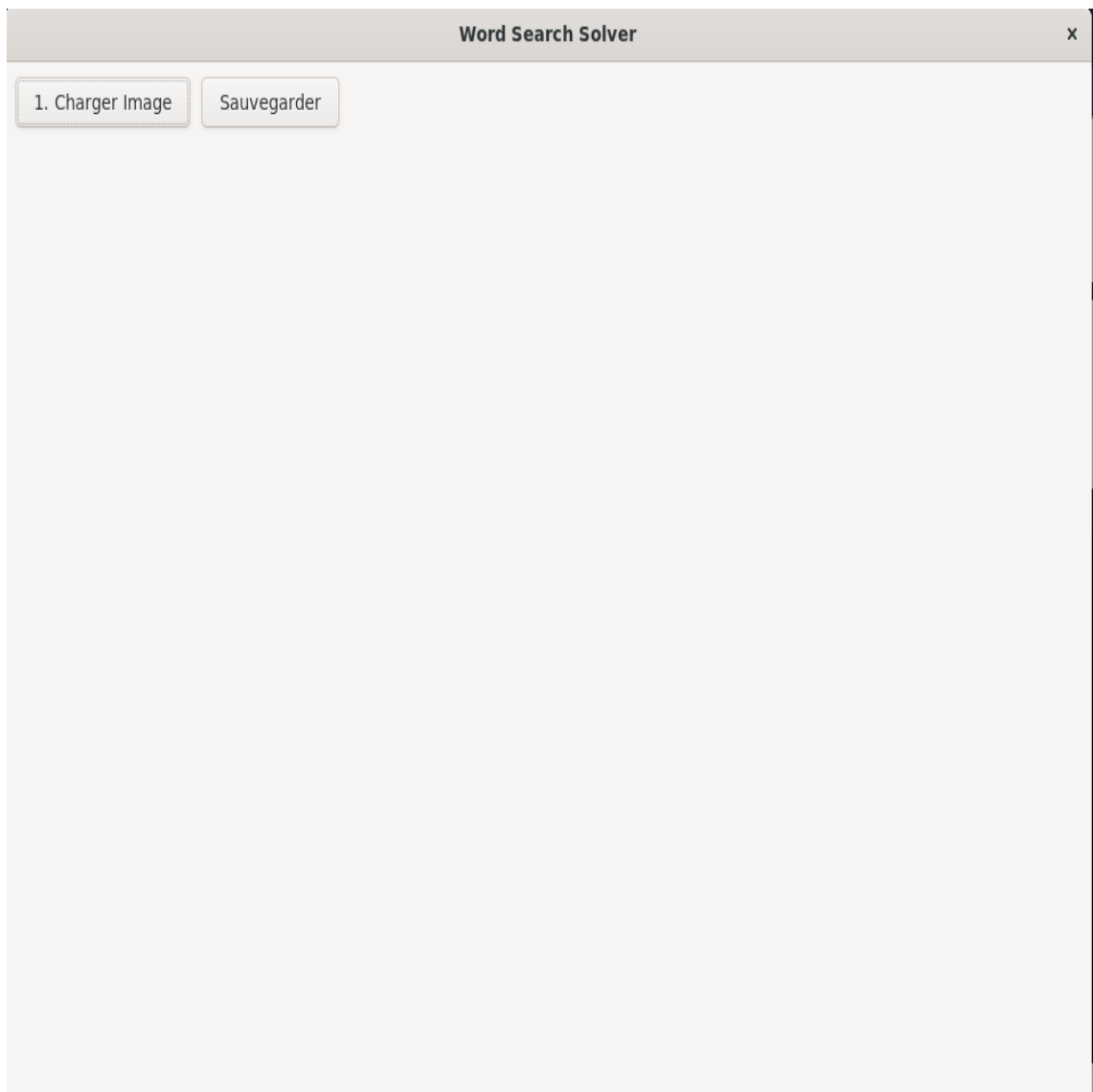


FIGURE 6 – Interface avant le chargement

4.5.2 Etape 2

Après avoir chargé l'image arrive le bouton prétraitement, celui-ci appliquera alors, comme son nom l'indique, toutes les fonctions relatives au prétraitement comme la rotation automatique, la mise en noir et blanc de l'image ainsi que le débruitage. Après cette phase, l'image apparaîtra donc prétraitée à l'écran.

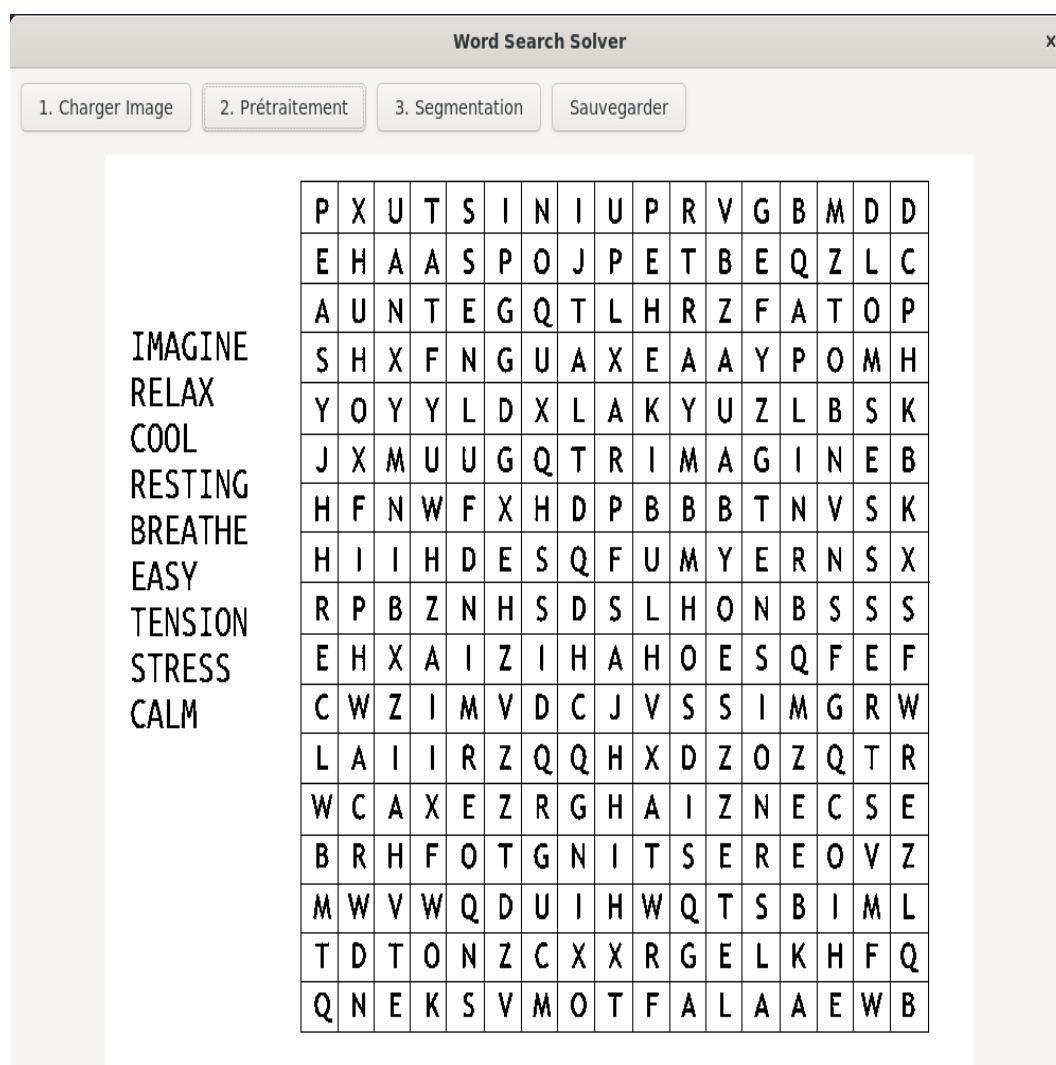


FIGURE 7 – Interface après le prétraitement

4.5.3 Etape 3

Ensuite encore un autre bouton, le bouton segmentation, celui-ci appliquera toutes les fonctions liées au découpage de l'image, donc les fonctions pour découper les lettres de la grille et la fonction pour découper chaque lettre de chaque mot de la liste de mots.

Une fois les fonctions appelées, cela écrira dans le terminal que les fichiers relatifs aux images ont bien été créés ainsi qu'une pop-up apparaîtra pour nous dire que le découpage s'est bien passé.

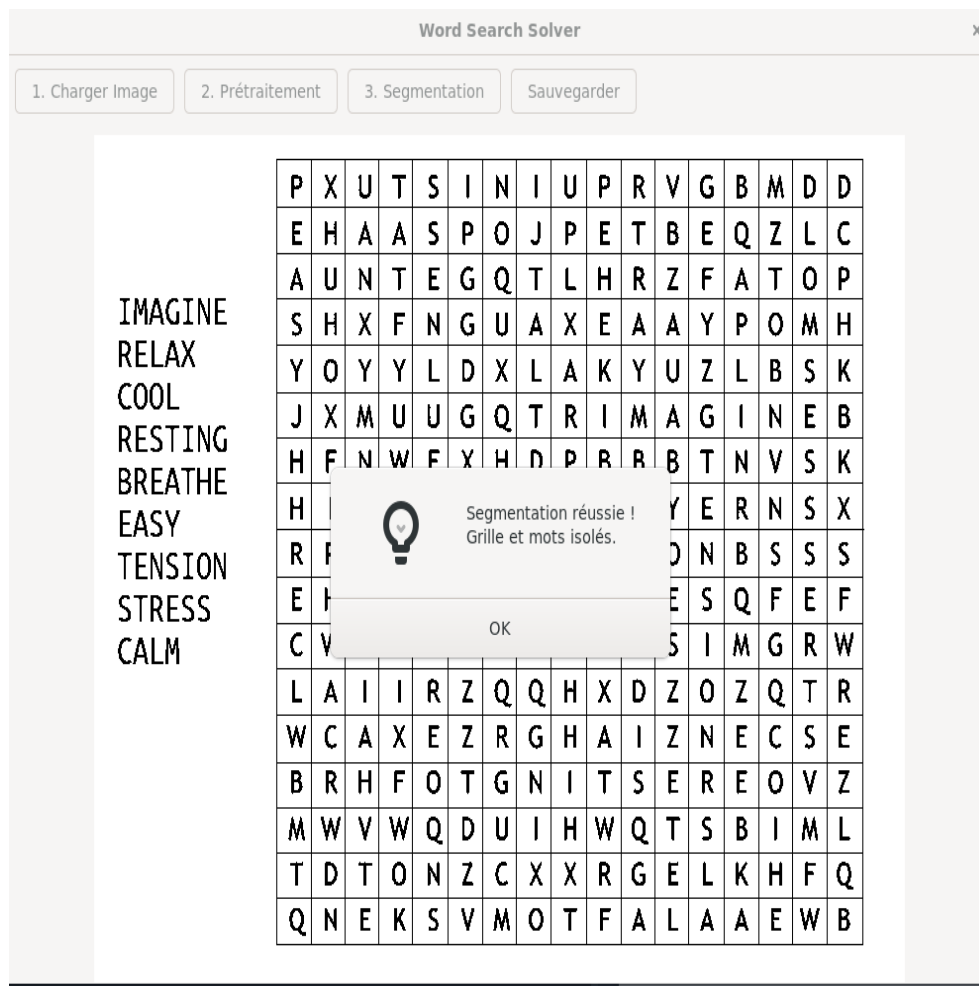


FIGURE 8 – Interface après la segmentation

4.5.4 Etape 4

Enfin le dernier bouton, mais pas des moindres, le bouton résolution, celui-ci appelle alors le réseau de neurones pour chaque image découpée et détecte la lettre, et ainsi on crée les fichiers 'grid.txt' et 'word.txt'.

A qui on envoie ensuite au solver qui nous donne les coordonnées de la lettre du début du mot et les coordonnées de la lettre de la fin du mot. On a plus qu'ensuite à tracer le trait sur la grille reconstruite et le tour est joué. Une

pop-up apparaît alors pour nous dire que l'OCR est réussie !

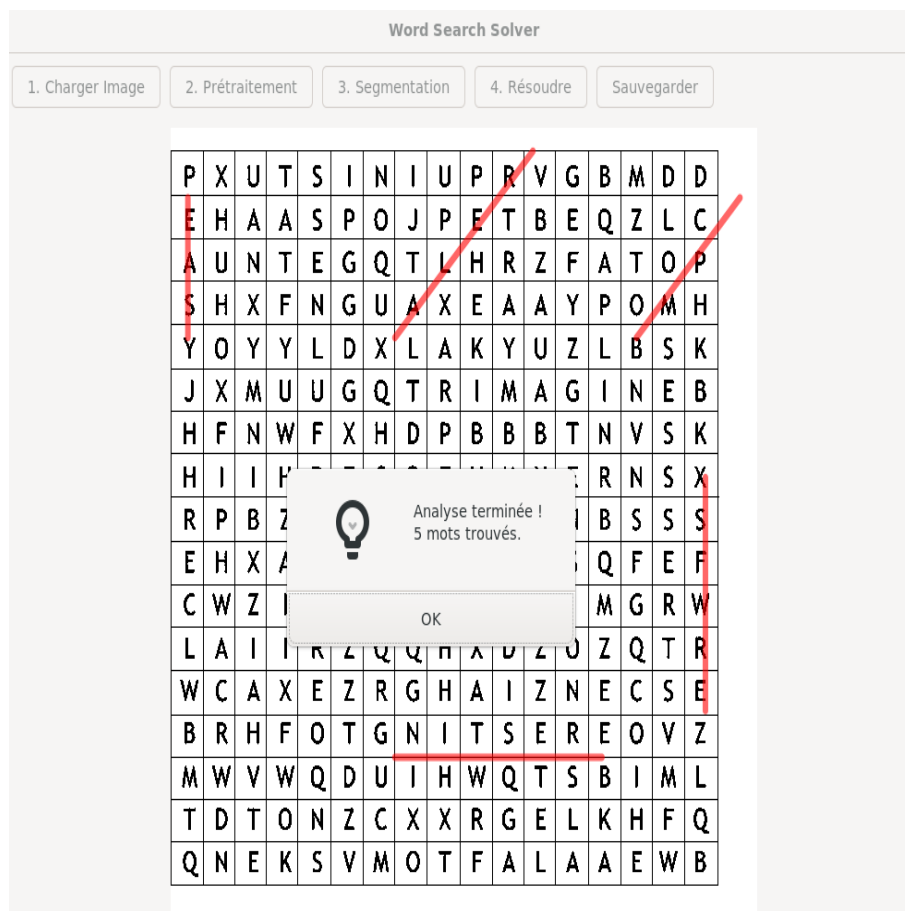


FIGURE 9 – Interface après la résolution

4.6 Makefile et gestion de la compilation

4.6.1 Présentation générale

Pour automatiser la compilation et l'exécution de notre projet OCR, nous avons conçu un *Makefile*. Celui-ci permet de :

- compiler automatiquement les fichiers source et générer les exécutables,
- gérer les dépendances entre fichiers pour ne recompiler que ce qui a changé,
- lancer des tests sans l'interface graphique facilement,
- nettoyer les fichiers intermédiaires et l'arborescence de build.

4.6.2 Organisation des cibles

Le Makefile comprend plusieurs cibles principales :

ocr Compile l'ensemble des sources avec l'interface graphique et génère l'exécutable **build/ocr**. Après compilation, le contenu de **README.md** est affiché.

test Compile l'ensemble des sources avec le fichier de test **test.c** et lance automatiquement le programme **build/test**.

build Crée le répertoire **build** si celui-ci n'existe pas, pour y stocker les exécutables.

all Compile à la fois l'interface graphique et le programme de test, générant **ocr** et **test** dans le répertoire courant.

clean Supprime le répertoire **build** et tous les fichiers intermédiaires, permettant une recompilation propre.

4.6.3 Gestion des fichiers source et des options de compilation

Le Makefile utilise les variables suivantes :

- **CC** : le compilateur utilisé (ici **gcc**),

- `CFLAGS` : options de compilation incluant l’affichage des avertissements (`-Wall`, `-Wextra`), le chemin vers les fichiers d’en-tête (`-Iinclude`), les bibliothèques GTK3 et la bibliothèque mathématique (`-lm`),
- `SRC` : la liste des fichiers sources du projet (`detect_cut.c`, `solver.c`, `image.c`, `ocr.c`).

Ces variables sont utilisées dans les différentes règles pour centraliser les paramètres de compilation et faciliter leur modification.

5 Conclusion

Ce projet d’OCR appliqué à la résolution automatique de grilles de mots cachés nous a permis de concevoir une application complète, combinant traitement d’image, reconnaissance de caractères, algorithmique et interface graphique. L’objectif principal, qui consistait à partir d’une image contenant une grille et une liste de mots pour produire une grille résolue automatiquement, a été globalement atteint pour les images correspondant aux niveaux 1 et 2 du cahier des charges.

L’ensemble de la chaîne de traitement a été implémenté avec succès : le prétraitement permet de corriger les imperfections des images d’entrée, la détection et le découpage isolent correctement les lettres et les mots, le réseau de neurones reconnaît les caractères majuscules de manière satisfaisante, et l’algorithme de résolution est capable de retrouver les mots dans la grille et d’en déterminer les coordonnées. L’interface graphique assure quant à elle une intégration cohérente de ces différents modules et facilite l’utilisation ainsi que la démonstration du projet.

Ce projet a également mis en évidence de nombreuses difficultés techniques, notamment liées au bruit des images, à la robustesse des algorithmes de segmentation et à la stabilité de l’entraînement du réseau de neurones. Leur résolution a nécessité des choix méthodologiques rigoureux, une bonne compréhension des concepts théoriques et une attention particulière portée aux détails d’implémentation, en particulier en ce qui concerne la gestion de

la mémoire et les aspects numériques.

Au-delà des aspects techniques, ce projet a constitué une expérience formatrice en termes de travail en équipe. La répartition des tâches, la communication régulière et l'intégration progressive des différentes composantes ont été des éléments clés de sa réussite. Il a permis à chaque membre du groupe de développer des compétences complémentaires tout en contribuant à un objectif commun.

Enfin, plusieurs pistes d'amélioration peuvent être envisagées, telles que l'augmentation de la robustesse du réseau de neurones face à des images plus dégradées, la prise en charge des niveaux supérieurs du cahier des charges ou encore l'optimisation des performances globales de l'application. Néanmoins, l'état actuel du projet constitue une base solide et fonctionnelle, répondant aux attentes fixées et démontrant la cohérence de l'approche adoptée.

6 Bibliographie

Voici ci dessous la liste des liens associées au differente ressource du projet :

6.1 Methode D'Otsu

https://fr.wikipedia.org/wiki/M%C3%A9thode_d%27Otsu

6.2 Transformée de Hough

https://fr.wikipedia.org/wiki/Transform%C3%A9e_de_Hough

6.3 Librairie Gtk

[https://fr.wikipedia.org/wiki/GTK_\(bo%C3%AEte_%C3%A0_outils\)](https://fr.wikipedia.org/wiki/GTK_(bo%C3%AEte_%C3%A0_outils))

6.4 Livre sur les neural networks

<http://neuralnetworksanddeeplearning.com/>

6.5 Documentation GdkPixBuf

<https://docs.gtk.org/gdk-pixbuf/index.html>